

Assignment 6 — Math Fundamentals for Robotics 16-811, Fall 2024

Mukai Yu
MSR, CMU
Pittsburgh, PA
mukaiy@andrew.cmu.edu

1. Implement two-dimensional (2D) convex hull.

Your algorithm should take as input a finite set of 2D points and produce as output a polygon constituting the boundary of the minimal convex set containing all the input points. (“minimal convex set” means that the set is a subset of any other convex set that contains all the input points.)

Solution:

```
def min_convex_set(points: np.ndarray) -> np.ndarray:
    """
    Compute the convex hull of a set of 2D points using Graham's scan algorithm.

    Args:
        points (np.ndarray): An array of shape (n, 2) containing 2D points.

    Returns:
        np.ndarray: An array of 2D points representing the vertices of the convex hull
                    in counterclockwise order.
    """
    # Input check
    assert points.shape[1] == 2, "Input points should be 2D"
    _points = points.copy().reshape(-1, 2)

    # Step 1: Find the bottom-most point (or the leftmost if there's a tie)
    bottom = _points[np.lexsort((_points[:, 0], _points[:, 1]))[0]]

    # Step 2: Sort the points by polar angle with respect to the bottom point
    def polar_angle(p):
        return np.arctan2(p[1] - bottom[1], p[0] - bottom[0])

    _points = sorted(_points, key=lambda p: (polar_angle(p), np.linalg.norm(p - bottom)))

    # Step 3: Initialize the convex hull with the first two points
    convex_hull = [_points[0], _points[1]]

    # Step 4: Iterate through the points to construct the convex hull
    for point in _points[2:]:
        while len(convex_hull) >= 2:
            # Check the orientation of the last three points
            p1, p2 = convex_hull[-2], convex_hull[-1]
            cross_product = (p2[0] - p1[0]) * (point[1] - p1[1]) - (p2[1] - p1[1]) * (point[0] - p1[0])
            if cross_product <= 0: # Remove the last point if it makes a non-left turn
                convex_hull.pop()
            else:
                break
        convex_hull.append(point)

    return np.array(convex_hull)
```

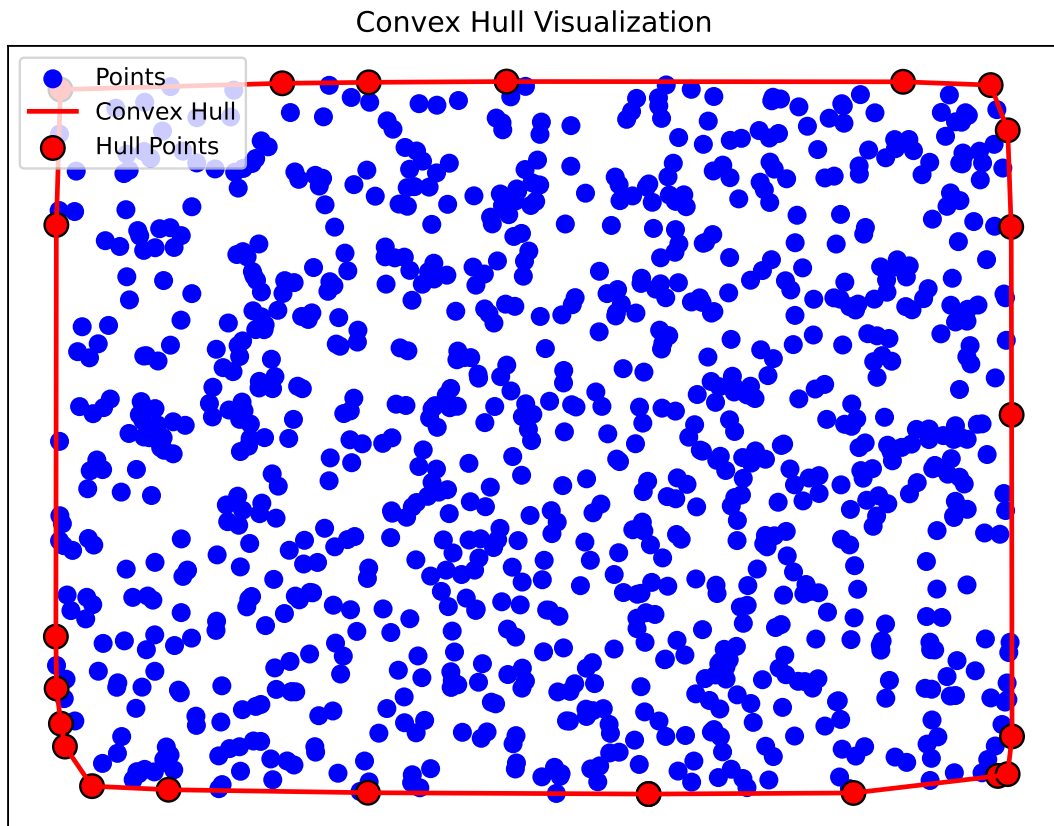


Fig. 1: Convex hull of a set of 2D points

2. Implement an algorithm for finding shortest paths in 2D polygonal environments.

The input to your algorithm should consist of a collection of nondegenerate convex polygons, a start location, and a goal location. The output should be a piecewise linear path that does not pass through the interior of any polygon and is as short as possible, if such a path exists. (Otherwise, your algorithm should indicate that no path exists.) One situation in which no path exists is if the start or goal is in the interior of a polygon.

Notes: (i) The polygons are allowed to overlap. (ii) The solution path is allowed to touch the boundaries of polygons, but is not allowed to pass through any polygon's interior.

Solution:

Procedure:

- Verify that the start and goal points are not inside any polygon, and if start – goal is a trivial solution
- Remove any polygon points if they are inside any other polygon
- Construct a visibility graph by connecting all pairs of valid vertices + start + goal, if the edge does not intersect any polygon (touching is allowed)
- Use Dijkstra's algorithm to find the shortest path

```
def find_shortest_path(start: np.ndarray, goal: np.ndarray, polygons: list) -> list:
    """
    Find the shortest path from start to goal avoiding polygon interiors.

    Args:
        start (np.ndarray): Start point of shape (2,).
        goal (np.ndarray): Goal point of shape (2,).
        polygons (list): List of np.ndarray, each of shape (n_i, 2), representing convex
            polygons.

    Returns:
        list: List of points representing the shortest path, or "No path exists".
    """
    # Verify that start and goal are not inside any polygon and trivial solutions
    if in_polygons(start, polygons) or in_polygons(goal, polygons):
        return "No path exists"
    if not line_intersects_polygons(start, goal, polygons):
        return [start, goal]

    # No trivial solution; construct a visibility graph
    G = nx.Graph()

    # Remove any polygon points if they are inside any other polygon
    valid_vertices = []
    for polygon in polygons:
        for vertex in polygon:
            if not in_polygons(vertex, polygons):
                valid_vertices.append(vertex)
        else:
            # Vertex is inside another polygon; do not include it
            pass

    # Include start and goal in the list of valid vertices
    all_vertices = [start, goal] + valid_vertices

    # Build the visibility graph by connecting all pairs of valid vertices
    for i, u in enumerate(all_vertices):
        u_tuple = tuple(u)
        G.add_node(u_tuple)
        for j in range(i + 1, len(all_vertices)):
            v = all_vertices[j]
            v_tuple = tuple(v)
            if not line_intersects_polygons(u, v, polygons):
                weight = np.linalg.norm(u - v)
                G.add_edge(u_tuple, v_tuple, weight=weight)

    # Use Dijkstra's algorithm from networkx
    try:
```

```

path = nx.dijkstra_path(G, tuple(start), tuple(goal), weight="weight")
path_coords = [np.array(node) for node in path]
return path_coords
except nx.NetworkXNoPath:
return "No path exists"

```

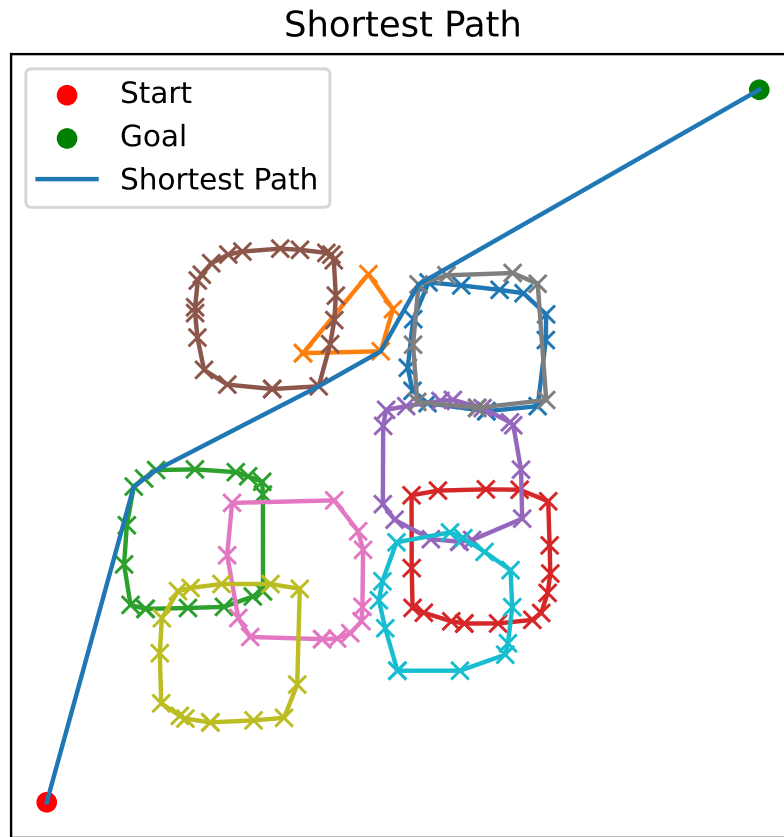


Fig. 2: Shortest path in a 2D polygonal environment

3. Combine your code for parts (1) and (2), along with some additional code, to implement shortest-path motion planning for convex polygons in two dimensions (translations only, no rotations). Specifically, the input to your algorithm should consist of a “robot” and its environment, along with start and goal configurations for the robot. The output should be a piecewise linear path as described below. (The robot moves only in straight lines. No rotations.)
- The robot should be a nondegenerate convex polygon. The environment should consist of a collection of nondegenerate (possibly overlapping) convex polygons. Given start and goal configurations of the robot, your algorithm should produce a shortest intersection-free path between the start and goal, if such a path exists. “Intersection-free” means that the robot may touch obstacle boundaries, even slide along obstacle boundaries, but may not touch the interior of any obstacle. If no such path exists, your algorithm should report that fact.

Solution:

This is basically constructing safe configuration space with respect to the robot’s anchor point then Q2.

Here’s how we can compute a safe configuration space for a robot with a given convex polygon obstacle using Minkowski sum:

```
def compute_safe_polygon(robot: np.ndarray, obstacle: np.ndarray) -> np.ndarray:
    """
    Compute the safe polygon of a robot with respect to an obstacle.
    The resulting polygon represents the Minkowski sum of the robot and the obstacle,
    which effectively enlarges the obstacle to account for the robot's shape.

    Args:
        robot (np.ndarray): (n, 2) vertices of the robot in counter-clockwise order
        obstacle (np.ndarray): (m, 2) vertices of the obstacle in counter-clockwise order

    Returns:
        np.ndarray: (k, 2) vertices of the safe polygon in counter-clockwise order
    """
    n = len(robot)
    m = len(obstacle)

    # Compute edge vectors for robot and obstacle
    E_p = [robot[(i + 1) % n] - robot[i] for i in range(n)]
    E_q = [obstacle[(i + 1) % m] - obstacle[i] for i in range(m)]

    # Compute angles of edge vectors
    angles_p = [np.arctan2(e[1], e[0]) for e in E_p]
    angles_q = [np.arctan2(e[1], e[0]) for e in E_q]

    # Normalize angles to [0, 2\pi)
    angles_p = [(a + 2 * np.pi) % (2 * np.pi) for a in angles_p]
    angles_q = [(a + 2 * np.pi) % (2 * np.pi) for a in angles_q]

    # Initialize indices and starting point
    i = j = 0
    s = robot[0] + obstacle[0]
    S = [s]

    while i < n or j < m:
        angle_p = angles_p[i % n] if i < n else float('inf')
        angle_q = angles_q[j % m] if j < m else float('inf')

        e_p = E_p[i % n] if i < n else np.array([0, 0])
        e_q = E_q[j % m] if j < m else np.array([0, 0])

        if abs(angle_p - angle_q) < 1e-10:
            # Angles are equal; advance both edges
            s = s + e_p + e_q
            i += 1
            j += 1
        elif angle_p < angle_q:
            # Advance robot edge
            s = s + e_p
```

```

        i += 1
    else:
        # Advance obstacle edge
        s = s + e_q
        j += 1

    S.append(s)

return np.array(S)

```

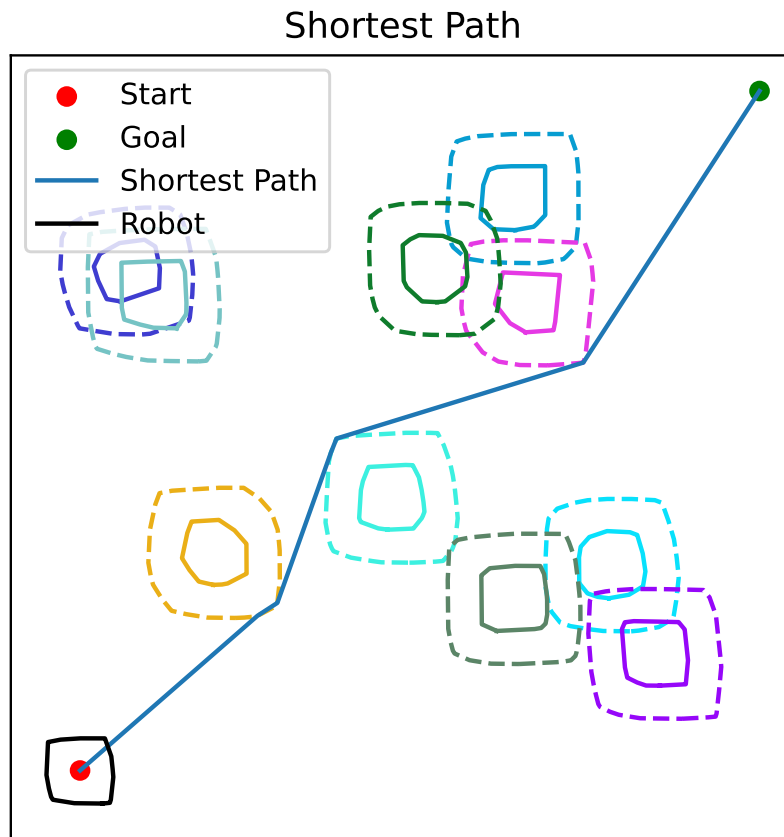


Fig. 3: Shortest path motion planning for convex polygons and convex robot